# Reflected XSS into HTML Context with All Tags Blocked Except Custom Ones

## 1. Overview

- **Vulnerability:** Reflected Cross-Site Scripting (XSS)
- **Category:** Web Application Security
- **Severity:** Medium to High (context-dependent)
- **Platform:** PortSwigger Web Security Academy
- **Vulnerability Type:** Client-side code injection
- **Status:** Successfully Exploited in a Controlled Lab Environment

## 2. Description

Reflected Cross-Site Scripting (XSS) occurs when an application includes user-controlled input in an HTTP response without applying appropriate output encoding or sanitization.

In this lab, the application implements a filtering mechanism that blocks standard HTML tags while allowing custom HTML elements. This restriction can be bypassed when the application fails to account for how browsers parse custom elements and process event-handler attributes.

An attacker may exploit this behavior to inject executable JavaScript into the rendered page, potentially executing it in the security context of the vulnerable website.

## 3. Vulnerability Details

The application reflects a user-controlled search query into the HTML response. Although standard HTML tags are blocked, the filtering mechanism does not sufficiently prevent the creation of custom elements with executable event-handler attributes.

This demonstrates that blacklist-based filtering is not a reliable defense against XSS. Blocking a predefined list of tags does not guarantee that injected content will remain harmless.

## 4. Lab Reproduction Steps

1. Open the assigned PortSwigger lab in a browser.
2. Locate the search functionality.
3. Submit a normal search query and observe how the input appears in the response.

4. Inspect the reflected input and identify the HTML context.
5. Test whether custom HTML elements are permitted by the filtering mechanism.
6. In the authorized lab environment, use the lab's intended custom-element and event-handler technique.
7. Verify whether JavaScript executes in the page context.
8. Confirm that the lab recognizes the vulnerability as solved.

**Expected result:** The custom-element filtering bypass allows JavaScript execution despite restrictions on standard HTML tags.

# 5. Security Impact

Successful exploitation may allow an attacker to:

- Execute JavaScript in a victim's authenticated browser session.
- Read sensitive information accessible to page scripts.
- Modify page content or manipulate user interactions.
- Perform unauthorized actions through the victim's browser, subject to application protections.
- Deliver convincing phishing content through a trusted website.

The actual impact depends on the application's authentication model, available browser APIs, Content Security Policy (CSP), and other security controls.

# 6. Root Cause

The primary root cause is insufficient validation and sanitization of user-controlled input before it is inserted into an HTML document.

The application relies on blocking known HTML tags instead of using context-aware output encoding or a robust HTML sanitization library.

# 7. Remediation

- Apply context-aware output encoding wherever untrusted input is rendered.
- Prefer safe DOM APIs such as `textContent` instead of inserting untrusted data through `innerHTML`.
- Avoid blacklist-based XSS filtering.
- If HTML input is a business requirement, use a maintained HTML sanitizer with a strict allowlist of permitted elements and attributes.
- Remove event-handler attributes from untrusted HTML.
- Deploy a restrictive Content Security Policy as an additional security layer.

- Add regression tests covering custom elements, event handlers, and unexpected HTML parsing behavior.

## 8. References

- PortSwigger Lab: https://portswigger.net/web-security/cross-site-scripting/contexts/lab-html-context-all-tags-blocked-except-custom-tags
- XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Cross-Site Scripting: https://owasp.org/www-community/attacks/xss/

## 9. Disclaimer

This report documents a controlled educational lab exercise. Testing should only be performed on systems for which explicit authorization has been granted.